

Database Record Layer on top of ToyDB

Utkarsh Shetye

Roll No: 24M2122

1 September 2024

Abstract

This comprehensive report details the design, architectural decisions, and full C implementation of a toy database system created for the CS631 course. The project involves bridging theoretical relational database concepts with low-level systems programming by constructing a custom record management layer over a raw paged file abstraction. Additionally, it features data ingestion, variable-length record serialization, and integration with a B-Tree access method for rapid querying capabilities. This document explores each system component deeply, featuring code walkthroughs, design tradeoffs, and a thorough evaluation of the system's correctness and functionality.

Contents

1	Introduction	3
2	System Architecture Overview	3
2.1	Physical File Layer (pflayer)	3
2.2	Access Method Layer (amlayer)	3
2.3	Record Layer (dblayer)	3
3	Task 1: Building a Record Layer	4
3.1	Slotted Page Architecture	4
3.2	Record Identification (RecId)	4
3.3	Table Initialization and Finalization	4
3.4	Insertion Logic: Table_Insert	4
3.5	Record Retrieval: Table_Get	6
3.6	Sequential Scanning: Table_Scan	7
4	Task 2: Loading CSV Data and Indexing	8
4.1	Schema Parsing and Data Encoding	8
4.2	Index Creation	8
5	Task 3: Retrieving Data	10
5.1	Decoding and Printing	10
5.2	Index Scanning	10
6	Conclusion	11

1 Introduction

The foundation of any modern relational database management system (RDBMS) relies on an efficient storage engine that maps logical schema structures to physical byte representations on a disk. The primary objective of this assignment was to construct exactly this mechanism: a functioning record layer on top of a physical page layer, loading data from a CSV file into this structure, and building mechanisms for efficient sequential and index-based retrieval.

The assignment was conceptually divided into three main tasks, each corresponding to a distinct phase of database execution:

1. **Building a Record Layer:** Designing a slotted-page structure to store and manage raw bytes (records) within fixed-size pages. This includes implementing insertion, fetching, and scanning algorithms.
2. **Data Loading and Indexing:** Reading schema and record information from a CSV file, encoding the data into binary format, inserting it into the database, and building a B-Tree index on a specified column to accelerate lookups.
3. **Data Retrieval and Verification:** Implementing sequential and index-scan capabilities to extract, decode, and display records, verifying data integrity against the original dataset.

The system is built using C and is structured into multiple interacting layers. Understanding how these layers communicate is crucial to understanding the database's overall design.

2 System Architecture Overview

To isolate complexity, the database system relies on a layered architecture that abstracts file operations and index management.

2.1 Physical File Layer (`pflayer`)

The lowest level of our architecture is the Physical File Layer. It provides a paged file abstraction. Instead of dealing with raw file descriptors and byte offsets, the higher layers treat a file as a collection of fixed-size pages (typically 4000 bytes). This layer handles caching, memory allocation, and flushing of these pages to the disk.

2.2 Access Method Layer (`amlayer`)

The Access Method Layer provides B-Tree index abstractions. It exposes functions such as `AM.InsertEntry` to add items to an index, and `AM.OpenIndexScan` alongside `AM.FindNextEntry` to facilitate efficient data retrieval based on search predicates.

2.3 Record Layer (`dblayer`)

The Record Layer (`dblayer`) is the core focus of this implementation. It manages the storage of tuples inside the pages provided by the `pflayer`. It is completely agnostic to the actual column schemas, treating records simply as variable-length byte arrays.

3 Task 1: Building a Record Layer

The core of the database engine is the record layer, implemented in `tbl.c` and `tbl.h`. The objective was to design a **slotted-page structure** to manage variable-length records within fixed-size pages.

3.1 Slotted Page Architecture

In a slotted page, records are stored starting from the end of the page (bottom-up), while the header information grows from the beginning of the page (top-down). The space in between is the free space.

The header implemented in `tbl.c` consists of:

- **Slot Count (2 bytes):** The number of records currently stored in the page.
- **Free Space Pointer (2 bytes):** The offset indicating the boundary of the free space.
- **Slot Array (Variable):** An array where each entry contains the offset (2 bytes) and the length (2 bytes) of a record.

3.2 Record Identification (RecId)

Tuples are uniquely addressed using a 4-byte Record Identifier (`RecId`).

- The first 3 bytes (24 bits) identify the **Page Number** within the database file.
- The last 1 byte (8 bits) identifies the **Slot Number** (the index in the slot array).

This bitwise combination (`pageNum << 16 | slot`) enables quick location of the physical bytes of any record.

3.3 Table Initialization and Finalization

The `Table_Open` function initializes the Physical Layer using `PF_Init()`, creates or opens the file (`data.db`), and prepares the `Table` struct with metadata. `Table_Close` ensures that any dirty pages are written back to the disk via `PF_UnfixPage` before closing the file descriptor.

3.4 Insertion Logic: Table_Insert

The `Table_Insert` function accepts a binary record buffer and its length, attempting to place it in the current page. If space is insufficient, a new page is allocated.

```
1 int Table_Insert(Table *tbl, byte *record, int len, RecId *rid) {
2     int fd=tbl->fd;
3     int pagenum;
4     char *buffer,*buffer2;
5     short offset,slotEntry;
6     char freeSpacePointer[2],offsetEntry[2],lengthEntry[2];
7     int previousPage;
8 }
```

```

9     if(tbl->currentPageNum==-1){
10         // Allocate first page if table is completely empty
11         int status=PF_AllocPage(fd,&pagenum,&buffer);
12         tbl->currentPageNum=pagenum;
13         tbl->bufferPointer=buffer;
14         setNumSlots(tbl->bufferPointer,0);
15     }
16     else{
17         // Check if current page has enough contiguous free space for
the new record
18         if((tbl->freeSpceEndLocator-tbl->slotEntryLocator+1)<(len+4)){
19             previousPage=tbl->currentPageNum;
20             // Allocate a new page and reset pointers
21             int status=PF_AllocPage(fd,&pagenum,&buffer2);
22             tbl->bufferPointer=buffer2;
23             tbl->currentPageNum=pagenum;
24             tbl->freeSpceEndLocator=FREE_SPACE_END_LOCATOR;
25             tbl->slotEntryLocator=SLOT_ENTRY_LOCATOR;
26             setNumSlots(tbl->bufferPointer,0);
27             // Flush the previous full page to disk
28             PF_UnfixPage(tbl->fd,previousPage,tbl->dirty);
29         }
30     }
31
32     // Calculate new record offset from the bottom of the page
33     offset=tbl->freeSpceEndLocator-len+1;
34     EncodeShort(offset,offsetEntry);
35     EncodeShort(len,lengthEntry);
36
37     // Write the actual record bytes into the free space
38     memcpy(tbl->bufferPointer+offset,record,len);
39     tbl->freeSpceEndLocator-=len;
40
41     // Update page header: Free space pointer, slot offset, and slot
length
42     EncodeShort((short)tbl->freeSpceEndLocator,freeSpacePointer);
43     memcpy(tbl->bufferPointer+FREE_SPACE_ENTRY_LOCATOR,
freeSpacePointer, 2);
44     memcpy(tbl->bufferPointer+tbl->slotEntryLocator, offsetEntry, 2);
45     memcpy(tbl->bufferPointer+tbl->slotEntryLocator+2, lengthEntry, 2);
46     tbl->slotEntryLocator+=4;
47
48     // Increment the total number of slots in the header
49     short slots=getNumSlots(tbl->bufferPointer);
50     slots++;
51     setNumSlots(tbl->bufferPointer,slots);
52
53     // Construct the 32-bit Record ID (PageNum + SlotNum)
54     *rid=tbl->currentPageNum<<16|slots;
55     tbl->dirty=1; // Mark page as dirty so it will be saved
56     return 0;
57 }

```

Listing 1: Table Insert Implementation

This logic ensures that records are placed sequentially at the bottom of the page, while the slot entries grow from the top.

3.5 Record Retrieval: Table_Get

Given a RecId, Table_Get parses out the Page Number and Slot Number, fetches the page, and retrieves the physical bits.

```
1 int Table_Get(Table *tbl, RecId rid, byte *record, int maxlen) {
2     char *buffer2;
3     // Extract Slot number (lower 16 bits) and Page number (upper 16
4     bits)
5     short slot = rid & 0xFFFF;
6     short pageNum = rid >> 16;
7     int noBytes;
8
9     // Fetch the specific page from the physical layer
10    int status=PF_GetThisPage(tbl->fd,pageNum,&buffer2);
11    if(status!=PFE_OK){
12        printf("Error in Getting This Page\n");
13        exit(EXIT_FAILURE);
14    }
15
16    // Parse the header to find where the record starts and its length
17    int slotOffset=getNthSlotOffset(slot,buffer2);
18    int slotLength=getLen(slot,buffer2);
19
20    // Safely copy the record bytes into the provided memory buffer
21    if(slotLength<=maxlen){
22        memcpy(record,buffer2+slotOffset,slotLength);
23        noBytes=slotLength;
24    }else{
25        memcpy(record,buffer2+slotOffset,maxlen);
26        noBytes=maxlen;
27    }
28
29    // Unpin the page so the buffer manager can evict it if needed
30    PF_UnfixPage(tbl->fd,pageNum,0);
31    return noBytes;
}
```

Listing 2: Table Get Implementation

This mechanism is extremely fast, requiring only straightforward pointer arithmetic after the page is pinned in memory by the pplayer.

3.6 Sequential Scanning: Table_Scan

The sequential scan function retrieves all records in the table, invoking a callback function for each.

```
1 void Table_Scan(Table *tbl, void *callbackObj, ReadFunc callbackfn) {
2     int fd=tbl->fd;
3     char *pageBuf,*actualAddress;
4     RecId recId;
5     int slots,pageNum;
6     short nthSlotOffset,nthSlotlength;
7     byte * row;
8
9     // Start scanning from the very first page of the file
10    int status=PF_GetFirstPage(fd,&(tbl->currentPageNum),&pageBuf);
11
12    while(status!=PFE_EOF){
13        slots=getNumSlots(pageBuf);
14        // Iterate through all valid slots in the current page header
15        for(int i=1;i<=slots;i++){
16            nthSlotOffset=getNthSlotOffset(i,pageBuf);
17            nthSlotlength=getLen(i,pageBuf);
18
19            // Locate the exact starting byte of the record
20            actualAddress=pageBuf+nthSlotOffset;
21
22            // Allocate memory and copy the record for safety
23            row=(char *)malloc(nthSlotlength*sizeof(char));
24            memcpy(row,actualAddress,nthSlotlength);
25
26            // Generate the RecId and invoke the user-provided callback
27            pageNum = tbl->currentPageNum << 16;
28            recId=pageNum|i;
29            callbackfn(callbackObj,recId,row,nthSlotlength);
30            free(row);
31        }
32        // Release the current page and move to the next logical page
33        PF_UnfixPage(tbl->fd,tbl->currentPageNum,0);
34        status=PF_GetNextPage(fd,&(tbl->currentPageNum),&pageBuf);
35    }
36 }
```

Listing 3: Table Scan Implementation

This algorithm effectively walks through the file page by page, resolving the tuples via the slot array.

4 Task 2: Loading CSV Data and Indexing

The `loaddb.c` file serves as the ingestion pipeline. It converts a text-based CSV dataset into our custom binary structure.

4.1 Schema Parsing and Data Encoding

The CSV's first line contains the schema. We parse this into a `Schema` struct. For each subsequent row, string data is split by commas. The `encode()` function translates strings to binary fields.

```
1 int encode(Schema *sch, char **fields, byte *record, int spaceLeft) {
2     int locator=0, len1;
3     int value;
4     long value2;
5     char * endptr;
6
7     // Iterate over each schema column to serialize the data
8     for(int i=0; i<sch->numColumns; i++){
9         if(sch->columns[i]->type==VARCHAR){
10             // Encode variable-length strings
11             len1=EncodeCString(fields[i], record+locator, spaceLeft);
12             locator+=len1;
13             spaceLeft-=len1;
14         }
15         else if(sch->columns[i]->type==INT){
16             // Convert string to 4-byte integer and encode
17             value= atoi(fields[i]);
18             EncodeInt(value, record+locator);
19             spaceLeft-=4;
20             locator+=4;
21         }
22         else if(sch->columns[i]->type==LONG){
23             // Convert string to 8-byte long and encode
24             value2= strtol(fields[i], &endptr, 10);
25             EncodeLong(value2, record+locator);
26             spaceLeft-=8;
27             locator+=8;
28         }
29     }
30     return locator; // Return total bytes used by the serialized record
31 }
```

Listing 4: Data Encoding Algorithm

This serialization logic guarantees that all column data is densely packed before insertion into the slotted page.

4.2 Index Creation

Simultaneously, a B-Tree index is initialized using `AM.CreateIndex`. As rows are inserted, the population column (an integer) is indexed against the `RecId`.

```
1 // Convert population string to integer
2 int population = atoi(tokens[2]);
3 // Encode integer into 4-byte index key format
4 EncodeInt(population, populationEntry);
```



```
5 // Insert the index key along with the associated RecId into the B-Tree
6 status=AM_InsertEntry(indexFd , 'i',4, populationEntry, rid);
```

Listing 5: Index Insertion Logic

This step allows logarithmic time complexity for filtering queries based on the population attribute.

5 Task 3: Retrieving Data

The `dumpdb.c` module validates that the data inserted can be successfully retrieved and decoded back to human-readable strings.

5.1 Decoding and Printing

The `printRow` callback handles the transformation from bytes back to typed primitives.

```
1 void printRow(void *callbackObj, RecId rid, byte *row, int len) {
2     int bufferReadPointer=0;
3     Schema *schema = (Schema *) callbackObj;
4     ColumnDesc * col;
5     printf("%10d ",(int)rid);
6
7     // Deserialize the raw bytes back into readable formats based on
8     Schema
9     for(int i=0;i<schema->numColumns;i++){
10         col=schema->columns[i];
11         if(col->type==VARCHAR){
12             // Decode string and advance the pointer
13             bufferReadPointer+=DecodeCString(row+bufferReadPointer,
14             buffer,MAX_PAGE_SIZE);
15             bufferReadPointer+=2;
16             printf("%-40s ",buffer);
17         }
18         else if(col->type==INT){
19             // Decode 4-byte integer and advance the pointer by 4 bytes
20             intData=DecodeInt(row+bufferReadPointer);
21             printf("%10d ",intData);
22             bufferReadPointer+=4;
23         }
24         else if(col->type==LONG){
25             // Decode 8-byte long integer and advance pointer by 8
26             bytes
27             longData=DecodeLong(row+bufferReadPointer);
28             printf("%20ld ",longData);
29             bufferReadPointer+=8;
30         }
31     }
32 }
```

Listing 6: Record Decoding and Formatting

5.2 Index Scanning

When retrieving data based on conditions (e.g., `population < 100000`), an index scan bypasses the sequential table scan.

```
1 void index_scan(Table *tbl, Schema *schema, int indexFD, int op, int
2     value) {
3     char valueBuffer[4];
4     EncodeInt(value,valueBuffer); // Serialize the comparison value
5
6     // Open a B-Tree index scan with the specified operator (e.g.,
7     LESS_THAN)
```

```

6  int scanDesc=AM_OpenIndexScan(indexFD,'i',4,op,valueBuffer);
7  int recId,length;
8  char buffer2[4095];
9
10 while(true){
11     // Fetch the next matching RecId from the B-Tree leaf nodes
12     recId=AM_FindNextEntry(scanDesc);
13     if(recId==AME_EOF) break;
14
15     // Fetch the actual record bytes from the heap file using the
16     RecId
17     length=Table_Get(tbl, recId, buffer2, MAX_PAGE_SIZE);
18
19     // Print the decoded record
20     printRow(schema,recId,buffer2,length);
21     printf("\n");
22 }
23 // Clean up index scan state
24 AM_CloseIndexScan(scanDesc);

```

Listing 7: Index Scan Implementation

This demonstrates the practical advantage of the Access Method layer, fetching exactly and only the pages and records required.

6 Conclusion

The assignment successfully demonstrates the implementation of a foundational database engine. By manually constructing a slotted-page architecture, the project provides a deep dive into how relational databases physically store tuples on disk. The integration with the `pplayer` and `amlayer` illustrates the synergy between buffer pooling, binary record storage, and B-Tree indexing.

The successful implementation of `tbl.c`, `loaddb.c`, and `dumpdb.c` ensures that data can be seamlessly converted from a text format to a highly optimized binary layout, persistently stored, and retrieved with full fidelity using both sequential and index-based execution paths. This system lays a robust groundwork for further database extensions, including dynamic schema evolution, a query parser, and comprehensive concurrency control protocols.

Github code:

<https://github.com/Utkarshshetye/Database-Record-Layer-on-top-of-ToyDB>